

Technical Report: Data-Driven Investment Intelligence Using NIFTY-50 Market Data

1. Executive Summary

This project implements an investment intelligence platform for the second Cult Open Projects 2026 problem statement. The system uses the provided NIFTY-50 historical market dataset to support data-driven investment decisions through stock forecasting, risk analytics, profile-based portfolio construction, explainable recommendations, anomaly detection, and an interactive dashboard.

The final prototype is not limited to stock-price prediction. It combines machine learning signals with financial risk measures so users can compare expected return, volatility, drawdown, and allocation rationale before making decisions.

2. Dataset and Exploratory Data Analysis

The project uses daily NIFTY-50 OHLCV records from 2000-01-03 to 2021-04-30. The loaded real dataset contains 235,192 price rows across 65 symbols. The data loader also reads `stock_metadata.csv` when present and merges company and industry information with the price records.

EDA focuses on date coverage, missing values, duplicate records, closing price trends, return distributions, trading volume behavior, volatility clustering, and maximum drawdowns.

Key EDA observations:

- The data contains changing NIFTY membership and legacy symbols, so not every stock has the same date range.
- Return distributions are noisy and heavy-tailed, which makes exact next-day return prediction difficult.
- Several stocks show large historical drawdowns, so risk-adjusted metrics are essential.
- Volume and return spikes provide useful signals for anomaly detection and explainability.

3. Feature Engineering

The feature pipeline converts raw OHLCV data into technical and statistical indicators. Features are computed independently for each symbol to avoid mixing information across stocks.

Implemented features:

- `return_1d`, `return_5d`, and `return_21d` for momentum.
- `ma_10_ratio` and `ma_30_ratio` for moving-average positioning.
- `ema_12_ratio` and `ema_26_ratio` for short and medium trend signals.
- `volatility_21d` for annualized rolling risk.
- `volume_zscore_21d` for unusual trading activity.
- `rsi_14` for overbought or oversold behavior.
- `macd` for trend and momentum.
- `bollinger_position` for price location inside rolling bands.

The prediction target is next-day return. A binary direction target is also generated to evaluate whether the model predicts upward or downward movement correctly.

4. Methodology

The platform follows a reproducible workflow:

1. Load all market CSV files from data/raw.
2. Skip non-price metadata files and merge metadata separately.
3. Normalize all price files into a common schema.
4. Engineer technical indicators for each stock.
5. Use a chronological train-test split per symbol.
6. Train a return forecasting model.
7. Evaluate model accuracy and directional behavior.
8. Generate latest stock forecasts.
9. Compute stock-level risk metrics.
10. Build investor-profile portfolios.
11. Generate explanations and anomaly flags.
12. Save reproducible CSV and model artifacts.

The train-test split uses the first 80 percent of each stock history for training and the final 20 percent for testing. This avoids look-ahead leakage and better reflects real investment use.

5. Model Architecture

The stock predictor engine uses ridge regression implemented with numpy. Ridge regression was chosen because it is transparent, fast, reproducible, and suitable as a baseline for noisy financial return data.

The model standardizes features, adds an intercept term, and solves the regularized least-squares equation. The saved model artifact includes feature names, means, scales, coefficients, and evaluation metrics in `models/ridge_return_model.json`.

This architecture is deliberately simple and explainable. In financial data, complex models can easily overfit historical noise, so a transparent baseline is useful for decision support and future comparison.

6. Experimental Results

Using the real NIFTY-50 CSV files, the balanced-profile pipeline produced:

- MAE: 0.0158
- RMSE: 0.0247
- R2: -0.0043
- Directional accuracy: 0.4973

The low R2 confirms that one-day return prediction is very difficult, which is expected for liquid equity markets. The platform therefore uses the model as one signal inside a broader risk-aware investment intelligence system rather than treating it as a standalone price oracle.

Balanced portfolio metrics:

- Annualized return: 13.10%
- Annualized volatility: 14.34%
- Sharpe ratio: 0.6350
- Maximum drawdown: -45.49%

The top balanced allocations include BHARTI, SSLT, BAJAJFINSV, JSWSTEEL, UTIBANK, TATASTEEL, HINDUNILVR, and NESTLEIND. These are selected through a combined score of forecasted return, Sharpe ratio, and volatility control.

7. Portfolio Construction Logic

The platform supports three investor profiles:

- Conservative: prioritizes lower volatility and lower concentration.
- Balanced: balances predicted return, Sharpe ratio, and volatility.
- Aggressive: gives more weight to expected return and allows higher concentration.

For each stock, the system computes a normalized score using predicted next-period return, historical Sharpe ratio, and a historical volatility penalty. Weights are normalized and profile-specific maximum weight caps are applied. This prevents excessive concentration and makes the output more practical for decision support.

8. Risk Assessment Methodology

Risk is evaluated at both stock and portfolio level. The implemented metrics are annualized return, annualized volatility, Sharpe ratio, Sortino ratio, maximum drawdown, portfolio-level annual return, portfolio volatility, portfolio Sharpe ratio, and portfolio drawdown.

The portfolio return calculation handles missing histories by treating unavailable daily returns as zero contribution for that date. This is important because the NIFTY-50 dataset includes changing index membership and legacy ticker symbols.

9. Explainability Techniques

Each portfolio allocation is accompanied by a natural-language explanation. The explanation includes investor profile, allocation weight, forecasted return direction, volatility, Sharpe ratio, and maximum drawdown.

Example:

BHARTI receives 2.4% in the balanced profile because it has a positive next-period signal, volatility of 44.8%, Sharpe ratio of 1.70, and max drawdown of -54.2%.

This makes the output transparent and helps users understand that recommendations are based on both opportunity and risk.

10. Market Anomaly Detection

The optional anomaly module identifies unusual market behavior using rolling z-scores for returns and volume changes. It flags sudden volatility spikes, extreme price movement, and unusual trading activity. These anomaly flags can help investors review periods where normal model assumptions may be less reliable.

11. Working Prototype

The working prototype is implemented as a Streamlit dashboard in app.py. It allows users to view historical stock performance, inspect latest forecasted opportunities, select investor profiles, view portfolio allocation charts, read explainable allocation notes, analyze risk metrics, and review detected market anomalies.

The application can be launched with:

```
streamlit run app.py
```

12. Key Insights and Future Work

Key insights:

- Short-horizon equity return prediction is noisy, so investment decisions should combine forecasts with risk analytics.
- Drawdown and volatility are crucial for practical investment intelligence.
- Transparent portfolio scoring is more useful for users than unexplained model output.
- Changing stock membership must be handled carefully in portfolio-level calculations.

Future improvements:

- Add walk-forward validation across rolling time windows.
- Compare ridge regression with random forests, gradient boosting, and LSTM models.
- Add sector-level diversification constraints.
- Include covariance-aware mean-variance optimization.
- Add feature attribution for deeper explainability.
- Add dashboard screenshots and hosted deployment for final presentation.

13. Deliverable Coverage

The repository satisfies the required deliverables:

- Working prototype: app.py Streamlit dashboard.
- Technical report: docs/REPORT.pdf.
- Public GitHub repository: source code, model artifact, configuration, documentation.
- README: setup, dependency installation, application launch, and reproducibility instructions.